

Multi-Tenant Log Monitoring Platform

System Design and SDLC Rules · Version 1.0 · October 8, 2026 · Status: Draft

1. Overview

Goal: a multi-tenant platform where any company can sign up, send logs from its apps, and get grouped errors, severity predictions, anomaly detection, AI explanations, and alerts.

Core principle: the automation core is NLP/AI (Python). The React website is only the interface that shows the results.

In scope (MVP): signup and organizations, API keys, log ingestion, search and filters, NLP severity and error grouping, LLM explanations, email and Slack alerts, basic dashboard, plans and quotas.

Out of scope (later): billing and payments, on-premise installs, custom ML per client, mobile app.

Success measures:

- Dashboard loads in under 2 seconds
- Recall on critical-severity logs above 90%
- Zero cross-tenant data leaks in isolation tests
- Ingestion handles the agreed target volume (to be confirmed before Step 2)

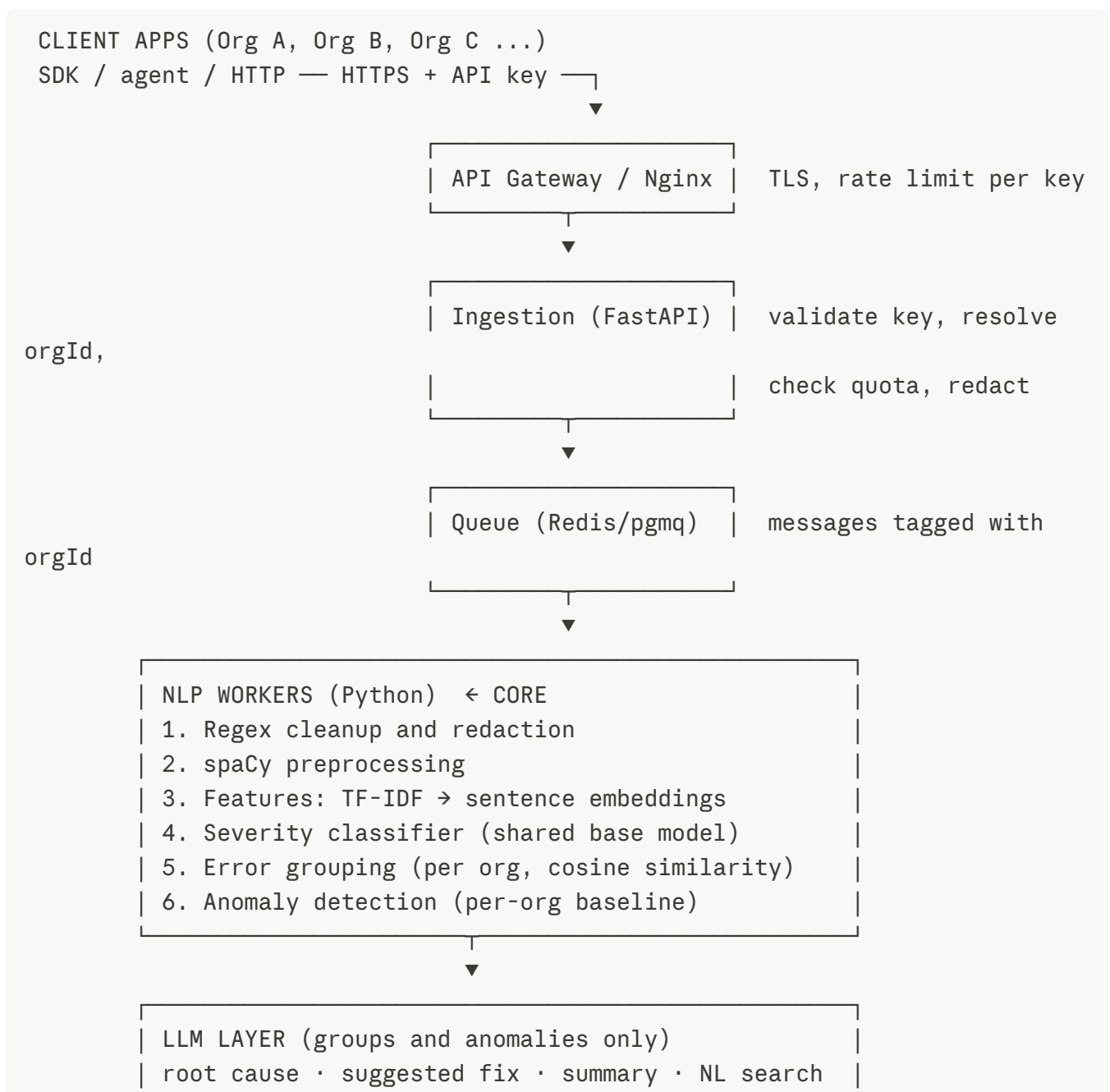
2. Tech Stack

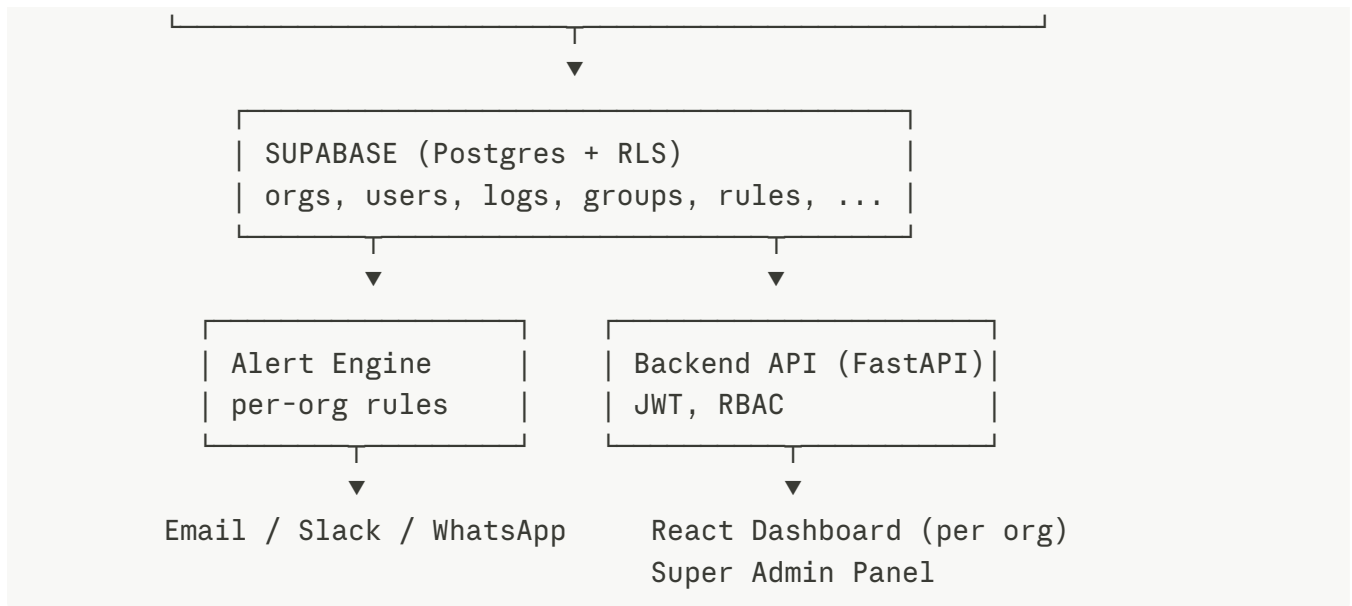
Layer	Choice	Reason
Database and auth	Supabase (Postgres)	Row Level Security enforces tenant isolation inside the database. Also provides auth, realtime, and storage.
Backend	FastAPI (Python)	One language for the API, ingestion, and NLP workers
Frontend	React	The only MERN part. Express and MongoDB are not used.
Realtime	Supabase Realtime	Live log stream on the dashboard
Queue	Redis (or pgmq for MVP)	Buffers traffic spikes

Layer	Choice	Reason
NLP	spaCy, scikit-learn, sentence-transformers	Core intelligence
LLM	Claude API (or another provider)	Explanations, summaries, natural-language search
Infra	Docker, Nginx, GitHub Actions	Deployment and CI/CD

3. System Design

3.1 Architecture





Data flow: app sends a log → ingestion validates and redacts → queue → NLP workers clean, classify, and group it → stored in Supabase → LLM explains new groups and anomalies → alert engine checks rules → dashboard updates live.

3.2 Tenant isolation (most important rule)

- Every table carries `org_id`.
- Supabase **Row Level Security** policies restrict every read and write to the user's own `org_id`. Isolation does not depend on developers remembering a filter.
- API keys map to exactly one organization.
- All indexes start with `org_id`, for example `(org_id, timestamp)`.
- Queue messages and cache keys are prefixed with `org_id`.
- Tenant isolation tests run in CI on every build.

3.3 Database schema

Table	Key fields
<code>organizations</code>	id, name, slug, plan_id, status, created_at, settings (retention, timezone)
<code>plans</code>	id, name, daily_log_quota, rate_per_min, max_users, retention_days, llm_budget
<code>users</code>	id, org_id, email, role (owner, admin, viewer), last_login
<code>api_keys</code>	id, org_id, service, key_hash, prefix, active, last_used

Table	Key fields
logs	id, org_id, timestamp, service, level, raw_message, clean_message, severity_predicted, confidence, group_id, anomaly_score, meta (jsonb), idempotency_key
groups	id, org_id, signature, sample_message, count, first_seen, last_seen, status, llm_summary, llm_status
alert_rules	id, org_id, condition, threshold, window_min, channels, cooldown_min, active
alert_history	id, org_id, rule_id, group_id, triggered_at, delivered
feedback	id, org_id, log_id, correct_label, user_id, created_at
usage_stats	org_id, date, log_count, rejected_count, llm_calls
audit_log	id, org_id, user_id, action, created_at

Scaling notes: partition `logs` by time, run a scheduled cleanup per plan retention, and keep an upgrade path to OpenSearch or ClickHouse if volume outgrows Postgres.

3.4 API design

Public (API key)

- `POST /v1/logs` (single or batch)

Dashboard (JWT)

- `POST /auth/signup`, `/auth/login`, `/auth/refresh`
- `GET /logs?level=&service=&from=&to=&q=&page=`
- `GET /groups`, `PATCH /groups/{id}` (resolve, ignore)
- `GET/POST/PUT/DELETE /alert-rules`
- `GET/POST/DELETE /api-keys`
- `GET/POST /users` (invite, change role)
- `POST /feedback`
- `POST /search/natural` (LLM-assisted search)
- `GET /stats/overview`, `GET /usage`

Super admin

- `GET /admin/orgs`, `PATCH /admin/orgs/{id}` (suspend, change plan)

- `GET /admin/system-health`

3.5 NLP pipeline (mapped to the NLP course)

Pipeline step	In this project
Data acquisition	Logs from the ingestion API
Text cleanup	Regex removes IDs, IPs, timestamps, paths; redacts emails, tokens, card numbers
Preprocessing	spaCy tokenization, lemmatization, stop-word removal
Feature engineering	TF-IDF first, then sentence embeddings
Model building	Severity classifier, error grouping, anomaly detection
Evaluation	Precision, recall, F1 on a labeled log set
Deployment	FastAPI workers in Docker
Monitoring and updating	User feedback and drift checks feed retraining

Multi-tenant NLP: a shared base model gives new clients value immediately. Error groups, anomaly baselines, and feedback corrections are stored per organization.

3.6 LLM layer

The LLM runs **only on error groups and anomalies**, never on every log.

1. **Root cause explanation** for each new error group
2. **Suggested fix** and plain-English summary
3. **Natural-language search:** "show payment errors from yesterday" becomes a safe, validated filter
4. **Alert text:** short readable messages instead of raw stack traces

LLM rules:

- Log content is untrusted input (prompt injection risk). It is passed as data only, and LLM output never triggers actions.
- Redact sensitive data before sending anything to the LLM.
- Cache results by group signature and enforce a per-org budget.
- Label output as "AI suggestion" and always show the source logs.
- If the LLM is unavailable, the system keeps working and the summary shows "pending".

3.7 Edge case register

Ingestion

- Invalid, expired, or revoked API key → 401
- Malformed JSON, missing fields, wrong timestamp format or timezone, future timestamps
- Oversized log lines (enforce a cap), binary data, non-English text and emoji
- Duplicate logs from client retries → idempotency key
- Flood from one client → rate limit and quota, return 429 with retry-after
- Client clock drift

Multi-tenant

- Org A attempting to read Org B data → blocked by RLS, covered by tests
- Suspended organization or exceeded quota
- Deleted organization → full data cleanup
- User removed from an org while a session is still active

NLP

- Empty or very short messages, unseen log formats
- Low-confidence severity → fall back to the level sent by the app, or mark "unknown"
- Sensitive data in logs → redact before storing
- Model drift when a client changes its log format

LLM

- API down, slow, or rate limited
- Wrong or invented explanations
- Prompt injection through log text
- Sensitive data leaking to the provider
- Cost overruns

Alerts

- Alert storms → deduplicate and apply a cooldown window
- Channel failure (email or Slack down) → retry and fallback channel
- Alert fires for a group already resolved

System

- NLP service down → logs are still stored and processed later from the queue
- Database slow or unavailable, partial failures

- Poison messages → dead-letter queue
- Table growth → partitioning and retention cleanup

3.8 Non-functional design

Concern	Design choice
Scalability	Stateless services behind a load balancer; scale NLP workers by queue depth
Fairness	Per-org rate limit and quota so one client cannot starve others
Reliability	Queue buffers when NLP is down; retries with a dead-letter queue
Security	HTTPS, hashed API keys, RBAC, RLS, audit log, secrets outside git
Retention	Per-plan retention with scheduled cleanup
Observability	Health checks, metrics, and logs for the platform itself
Backup	Daily backups with a tested restore

4. SDLC Rules

4.1 Model

Agile (Scrum-lite) with 2-week sprints, preceded by a short upfront design phase.

4.2 Phase gates

Phase	Entry	Activities	Exit criteria	Deliverables
1. Planning	Idea approved	Goals, target clients, plans, risks, timeline	Scope signed off	Project charter, risk register
2. Requirements	Charter approved	Interviews with 2-3 pilot clients, user stories, acceptance criteria	Requirements baselined	SRS, prioritized backlog (MoSCoW)

Phase	Entry	Activities	Exit criteria	Deliverables
3. Design	SRS approved	Architecture, schema, API spec, NLP and LLM design, wireframes, threat model	Design review passed	Design doc, OpenAPI spec, ER diagram, wireframes
4. Implementation	Design approved	Sprints	Sprint goals and Definition of Done met	Working increments
5. Testing	Feature complete	Unit, integration, load, security, tenant-isolation, NLP evaluation, UAT	No open critical or high bugs	Test report
6. Deployment	Test report approved	Staging, then production, migrations, rollback plan	Smoke tests pass in production	Release notes, runbook
7. Maintenance	Live	Monitoring, retraining, bug fixes, feature requests	Ongoing	Monthly report

4.3 Sprint plan

Sprint	Goal
0	Repo, CI/CD, Docker, Supabase project, orgs, users, auth, RLS policies, isolation tests
1	API keys, ingestion API, queue, store logs
2	Dashboard: live stream, filters, search
3	NLP v1: regex cleanup, preprocessing, TF-IDF error grouping
4	Severity classifier and evaluation
5	LLM layer: group explanations, natural-language search

Sprint	Goal
6	Alert engine and channels
7	Plans, quotas, rate limits, usage page
8	Embeddings and anomaly detection
9	Super admin panel, charts, hardening, load test
10	Pilot with real clients, fixes, launch

4.4 Version control

- Branches: `main` (production), `develop` (staging), `feature/*`.
- Every change goes through a pull request with at least 1 review and passing CI.
- Commit style: `feat:`, `fix:`, `docs:`, `test:`, `chore:`.
- No direct pushes to `main` or `develop`.

4.5 Definition of Done (per task)

1. Code reviewed and merged
2. Unit tests written and passing
3. Tenant isolation verified if the task touches data
4. Edge cases from the register considered and tested
5. No secrets in code or logs
6. API docs updated
7. Deployed to staging and checked

4.6 Testing protocol

- **Unit:** pytest (backend and NLP), Jest or Vitest (React). Target 70%+ coverage on core logic.
- **Integration:** ingestion → queue → NLP → database → alerts.
- **Tenant isolation:** Org A must never read, write, or trigger alerts on Org B data. Runs in CI.
- **Load:** simulate 10x expected volume (k6 or Locust).
- **NLP evaluation:** labeled log set with precision, recall, and F1. A new model ships only if it beats the current one and keeps a recall floor on critical logs.
- **LLM evaluation:** review a sample of explanations for accuracy, test prompt-injection cases, test failure and timeout handling.

- **Security:** dependency scan, OWASP Top 10 checklist, API key and JWT tests.
- **UAT:** pilot clients sign off.

4.7 Security protocol

- Secrets in environment variables or a vault, never in git.
- Least-privilege database roles.
- Validate input on every endpoint.
- Hash passwords and API keys.
- Audit log for sensitive actions.
- Delete all data when an organization is removed.

4.8 Release protocol

- Semantic versioning.
- Staging deploy, smoke tests, then production.
- Database migrations must be backward compatible.
- A rollback plan is prepared before every release.
- Release notes for clients.

4.9 Change management

Client requests go into the backlog, get estimated, and enter a sprint. No scope changes mid-sprint except critical bugs.

4.10 Incident protocol

- Severity levels: P1 (down), P2 (degraded), P3 (minor).
- Steps: acknowledge, mitigate, fix, then a short post-mortem.

4.11 NLP and LLM model protocol

- Version every model and dataset.
- Shadow-test new models on live traffic before switching.
- Monitor drift (share of unknown or low-confidence logs) and retrain when it rises.
- Track LLM cost, latency, and error rate per organization.
- Prompts are versioned and reviewed like code.

4.12 Documentation protocol

Keep current: README, API spec, architecture doc, runbook, and client onboarding guide.

5. Step-by-Step Roadmap

1. Requirements and scope
2. Architecture and database schema (Supabase and RLS)
3. API spec (FastAPI)
4. Tenant auth and isolation
5. Ingestion and queue
6. NLP pipeline
7. LLM layer
8. Alerts
9. Dashboard
10. Testing, deployment, and maintenance

Next action: confirm the target volume for version 1 (number of clients and logs per day per client), then finalize the requirements document (Step 1).